

Finite Automata

1. Introduction to Machines and Automata

1.1 What is a Machine?

Definition: In general, a machine is something that:

- Takes **input**
- **Processes** it according to some rules
- Produces an **output**

In Finite Automata Context: A machine is a mathematical model that:

- Reads a sequence of symbols (input string)
- Decides whether to **accept** or **reject** the string

1.2 What is an Automaton?

Etymology: The word "automaton" (plural: automata) comes from Greek, meaning "something that moves by itself"

Definition: An automaton is a kind of machine that automatically changes its **state** when it reads inputs.

In Computer Science: Automata are abstract machines used to model how computers or systems work.

1.3 What is a Finite Automaton?

Definition: A finite automaton is a special type of automaton that has a **finite (limited) number of states**.

Key Characteristics:

- Can only be in **one state at a time** (for DFA)
- Changes states based on input symbols
- Used to recognize patterns
- Decides if a string belongs to a particular language

Purpose: Used for pattern recognition and language acceptance.

2. Fundamental Concepts

2.1 State

Definition: A state represents a particular condition or situation of the machine.

Key Point: At any moment, the machine is in **exactly one state**.

Real-World Examples:

EXAMPLE 1: Traffic Light

States: "Red", "Green", "Yellow"

- At any time, light is in one state
- Transitions happen based on timer

EXAMPLE 2: Door Lock

States:

- "Locked"
- "Unlocked"

Notation: States are usually written as: $q_0, q_1, q_2, q_3, \dots$

Visual Representation:

State in Diagram:

$(q_0) \leftarrow$ Circle represents a state

Start State:

$\rightarrow(q_0) \leftarrow$ Arrow pointing to start state

Final/Accepting State:

$((q_2)) \leftarrow$ Double circle for accepting state

2.2 Input Alphabet (Σ)

Definition: The alphabet is the set of all symbols that the machine can read as input.

Notation: Denoted by Σ (sigma)

Examples:

EXAMPLE 1: Binary strings

$\Sigma = \{0, 1\}$

EXAMPLE 2: English lowercase letters

$\Sigma = \{a, b, c, d, \dots, z\}$

EXAMPLE 3: Decimal digits

$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

EXAMPLE 4: Simple arithmetic

$\Sigma = \{+, -, *, /, 0, 1, 2, \dots, 9\}$

2.3 Types of Finite Automata

Two Main Types:

TYPE 1: Deterministic Finite Automata (DFA)

- The machine can exist in ONLY ONE state at any given time
- Each transition is deterministic (predictable)
- For each state and input, exactly one next state

TYPE 2: Non-deterministic Finite Automata (NFA)

- The machine can exist in MULTIPLE states at the same time
- Some transitions can be non-deterministic
- For each state and input, zero or more next states possible

3. Deterministic Finite Automata (DFA)

3.1 Formal Definition

A DFA consists of five components:

COMPONENT 1: Q

- A finite set of states
- Example: $Q = \{q_0, q_1, q_2\}$

COMPONENT 2: Σ

- A finite set of input symbols (alphabet)
- Example: $\Sigma = \{0, 1\}$

COMPONENT 3: q_0

- The start state (initial state)
- Must be an element of Q
- Example: q_0

COMPONENT 4: F

- A set of final (accepting) states
- Subset of Q ($F \subseteq Q$)
- Example: $F = \{q_2\}$

COMPONENT 5: δ

- Transition function
- Mapping: $Q \times \Sigma \rightarrow Q$
- For each state and symbol, gives exactly one next state

Formal Notation: A DFA is defined by the 5-tuple:

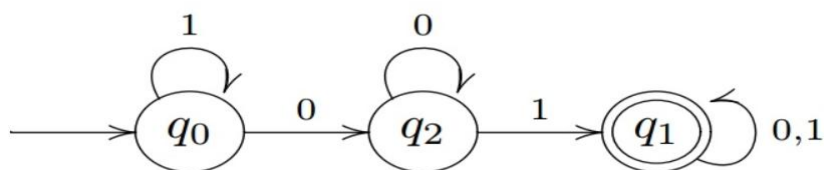
$$\text{DFA} = (Q, \Sigma, q_0, F, \delta)$$

3.2 Graphical Representation of DFA

Components in State Diagram:

NODES = States

- Drawn as circles
- Start state: has incoming arrow labeled "Start"
- Final states: drawn as double circles



Components:

- q_0 : Start state (single circle with start arrow)
- q_1 : Final state (double circle)

3.3 How DFA Processes Input

Algorithm:

INPUT: A word w in Σ^* (string of symbols)

QUESTION: Is w acceptable by the DFA?

STEPS:

STEP 1: Start at the start state q_0

STEP 2: Read the input string one symbol at a time

STEP 3: For each symbol:

- Look at current state
- Look at current symbol
- Use transition function δ to move to next state

STEP 4: After reading all symbols:

- If current state is in F (final states): ACCEPT
- Otherwise: REJECT

Example Execution:

DFA: States = $\{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

Start = q_0

Final = $\{q_1\}$

Input: $w = "01"$

Execution:

- Position: Start at q_0
- Read '0': $\delta(q_0, 0) = q_2$ (move to q_2)
- Read '1': $\delta(q_2, 1) = q_1$ (move to q_1)
- End: Current state = q_1
- q_1 is in F ? YES

Result: **ACCEPT** ✓

Input: $w = "10"$

Execution:

- Position: Start at q_0
- Read '1': $\delta(q_0, 1) = q_0$ (stay at q_0)
- Read '0': $\delta(q_0, 0) = q_2$ (move to q_2)

- End: Current state = q_2
- q_2 is in F ? NO

Result: **REJECT** X

4. Regular Languages

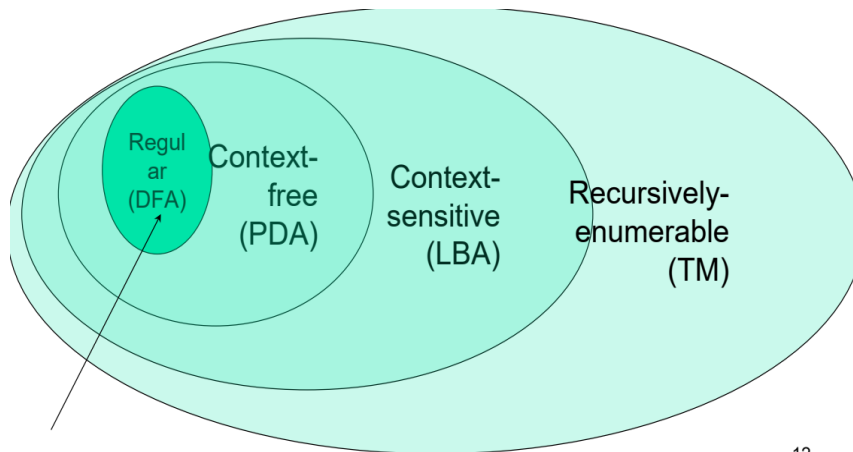
4.1 Definition

Regular Language: Let $L(A)$ be a language recognized by a DFA A . Then $L(A)$ is called a "Regular Language".

Simple Explanation: Any language that can be recognized by a DFA is a regular language.

4.2 Chomsky Hierarchy

The Hierarchy of Language Classes:



12

5. DFA Examples

5.1 Example 1: Strings Containing "01"

Problem: Build a DFA for the language:

$L = \{w \mid w \text{ is a binary string that contains "01" as a substring}\}$

Regular Expression: $(0+1)^*01(0+1)^*$

Solution Steps:

STEP 1: Define alphabet

$\Sigma = \{0, 1\}$

STEP 2: Design states

$Q = \{q_0, q_1, q_2\}$

- q_0 : Haven't seen anything useful yet

- q_1 : Just saw a '0'
- q_2 : Saw "01" (accepting state)

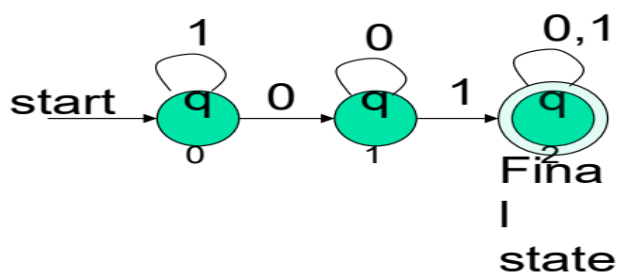
STEP 3: Define start and final states

Start state: q_0

Final states: $F = \{q_2\}$

STEP 4: Design transitions

State Diagram:



Transition Table:

		symbols	
		0	1
states	δ q_0	q_1	q_0
	q_1	q_1	q_2
	$*q_2$	q_2	q_2

State Meanings:

- q_0 : Initial state or just saw a '1'
- q_1 : Just saw a '0', waiting for '1'
- q_2 : Saw "01", stay here forever (accepting)

Test Cases:

TEST 1: Input = "01"

Path: $q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2$

Result: ACCEPT ✓

TEST 2: Input = "10"

Path: $q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1$

Result: REJECT $\times$ (ended in q_1 , not final)

TEST 3: Input = "1101"

Path: $q_0 \xrightarrow{1} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2$

Result: ACCEPT $\checkmark$

What Makes This Deterministic:

- From each state, for each input symbol (0 or 1), there is **exactly one** transition
- No ambiguity about which state to go to

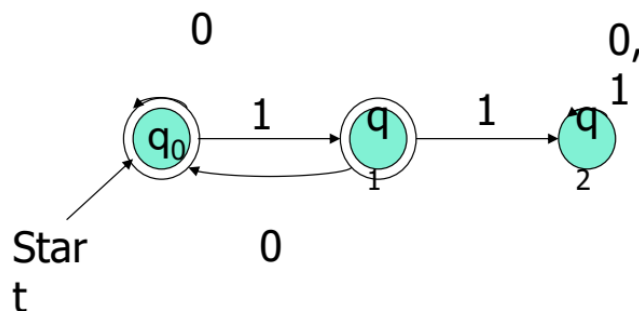
5.2 Example 2: No Two Consecutive 1's

Problem: DFA that accepts all strings without two consecutive 1's.

Language: $L = \{\epsilon, 0, 1, 00, 01, 10, 000, 001, 010, 100, 101, \dots\}$

- Accepts: "", "0", "1", "01", "10", "010"
- Rejects: "11", "011", "110", "111"

Solution:



• $Q = \{q_0, q_1, q_2\}$

• $\Sigma = \{0, 1\}$

• start state = q_0

• $F = \{q_0\}$

• Transition table

		symbols	
		0	1
states	q_0	q_0	q_1
	q_1	q_0	q_2
	$*q_2$	q_2	q_2

State Meanings:

- q_0 : Safe state (last symbol was 0 or start)
- q_1 : Just saw one '1' (careful!)

- q_2 : Saw "11" - dead state (reject forever)

Test Cases:

TEST 1: Input = "010"

Path: $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0$

Result: ACCEPT ✓

TEST 2: Input = "11"

Path: $q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_2$

Result: REJECT ✗

5.3 Example 3: Substring "11" (Clamping Logic)

Problem: Build a DFA that recognizes strings containing "11" as a substring (clamping circuit - once it sees "11", it stays "on").

Language: $L = \{w \mid w \text{ is a bit string which contains substring "11"}\}$

State Design:

- q_0 : Start state (never seen "11", last input wasn't '1')
- q_1 : Saw one '1', waiting for another
- q_2 : Saw "11" at least once (clamp on forever)

State Diagram:

Start

↓

$(q_0) \xrightarrow{1} (q_1) \xrightarrow{1} (q_2)$ (wrong maybe)

↑ ∪ ↑ ∪ ↑ ∪

| 0 | 0 | 0,1

Transition Table:

State	0	1
q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_2	q_2

Test Cases:

TEST 1: Input = "0110"

Path: $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_2$

Result: ACCEPT ✓

TEST 2: Input = "101"

Path: $q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_0 \xrightarrow{1} q_1$

Result: REJECT ✗

5.4 Example 4: Even Number of 0s and 1s

Problem: Build a DFA for:

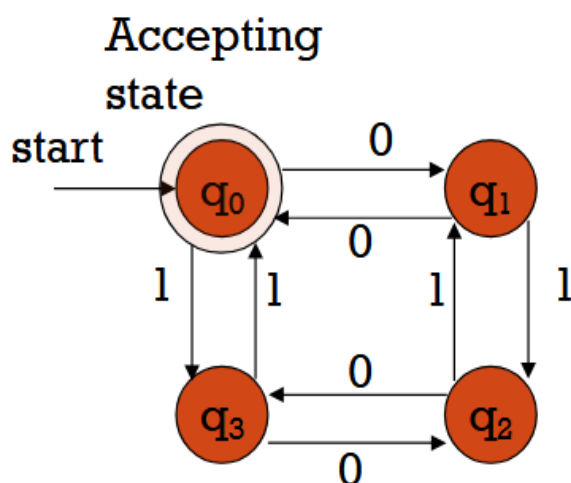
$L = \{w \mid w \text{ is a binary string that has even number of 1s AND even number of 0s}\}$

Key Insight: Need to track parity of both 0s and 1s.

States (4 needed):

- q_0 : Even 0s, Even 1s (ACCEPTING)
- q_1 : Even 0s, Odd 1s
- q_2 : Odd 0s, Even 1s
- q_3 : Odd 0s, Odd 1s

State Diagram:



Transition Table:

State	0	1
q ₁	q ₃	q ₂
q ₂	q ₄	q ₁
q ₃	q ₁	q ₄
q ₄	q ₂	q ₃

Test Cases:

TEST 1: Input = "" (empty)

State: q₀₀

Result: ACCEPT ✓ (0 is even)

TEST 2: Input = "01"

Path: q₀₀ --0--> q₁₀ --1--> q₁₁

Result: REJECT X

6. Extended Transition Function ($\hat{\delta}$)

6.1 Definition

In automata theory, the Extended Transition Function, denoted by $\hat{\delta}$ or δ^* , determines the state an automaton reaches after processing an entire string of symbols, rather than a single input symbol. **Purpose:** Extend δ from single symbols to entire strings.

Notation: $\hat{\delta}(q, w)$ = destination state from state q on input string w

Formal Definition:

BASIS: $\hat{\delta}(q, \epsilon) = q$

(Reading empty string leaves you in same state)

INDUCTION: $\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$

where w is a string and a is a single symbol

Simple Explanation:

- To process string wa (w followed by a):

- First process w to get to some state
- Then process symbol a from that state

6.2 Example Calculation

Given DFA:

Transition Table:

State	0	1
A	A	B
B	A	C
*C	C	C

Calculate: $\hat{\delta}(B, 011)$

METHOD 1: Step-by-step

$$\begin{aligned}
 \hat{\delta}(B, 011) &= \delta(\hat{\delta}(B, 01), 1) \\
 &= \delta(\delta(\hat{\delta}(B, 0), 1), 1) \\
 &= \delta(\delta(\delta(B, 0), 1), 1) \\
 &= \delta(\delta(A, 1), 1) \quad [\text{From the table } (B,0)=a] \\
 &= \delta(B, 1) \\
 &= C
 \end{aligned}$$

METHOD 2: Direct trace

Start: B

Read '0': B --0--> A

Read '1': A --1--> B

Read '1': B --1--> C

End: C

Result: $\hat{\delta}(B, 011) = C \checkmark$

7. Language of a DFA

7.1 Definition

Formal Definition:

$$L(A) = \{w \mid \delta(q_0, w) \in F\}$$

In Plain English: The language of DFA A is the set of all strings w such that starting from q_0 and reading w leads to a state in F (final states).

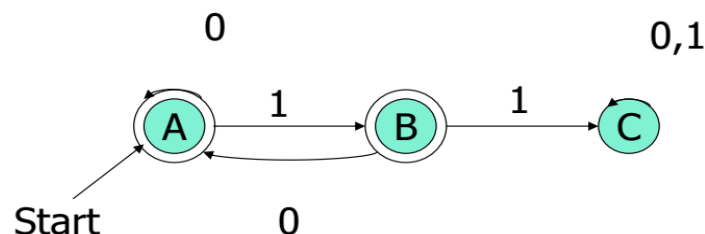
Alternative Definition:

$L(A)$ = All strings that lead from q_0 to a final state

7.2 Example

DFA:

State Diagram:



This DFA accepts strings in:

$$L(A) = \{w \mid w \text{ is in } \{0,1\}^* \text{ and } w \text{ does NOT have two consecutive 1's}\}$$

Accepted: "", "0", "1", "00", "01", "10", "000", "001", "010", "100", "101"

Rejected: "11", "011", "110", "111", "1100", "0111"

8. Non-Deterministic Finite Automata (NFA)

8.1 Definition and Characteristics

Key Difference from DFA: An NFA can exist in **multiple states simultaneously**.

Non-determinism means:

- For a given state and input symbol, there may be **zero, one, or multiple** possible next states
- The machine explores all possibilities in parallel

Formal Definition:

An NFA consists of:

1. Q : Finite set of states
2. Σ : Finite set of input symbols
3. q_0 : Start state
4. F : Set of final states
5. δ : Transition function

$\delta: Q \times \Sigma \rightarrow P(Q)$ [Power set of Q]

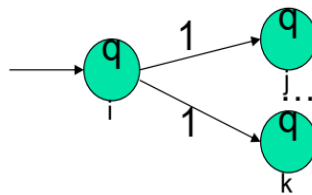
Maps to a SET of states (not just one)

Notation: $NFA = (Q, \Sigma, q_0, F, \delta)$

2^Q combination Possible.

Visual Example of Non-determinism:

From state q_i with input '1':



Both transitions are valid!

Machine can be in $\{q_j, q_k\}$ simultaneously

8.2 How to Use an NFA**Algorithm:**

INPUT: A word w in Σ^*

QUESTION: Is w acceptable by the NFA?

STEPS:

STEP 1: Start with current states = $\{q_0\}$

STEP 2: For each symbol in w :

- For each current state
- Find all possible next states for this symbol
- Union all these next states
- This becomes new set of current states

STEP 3: After consuming all symbols:

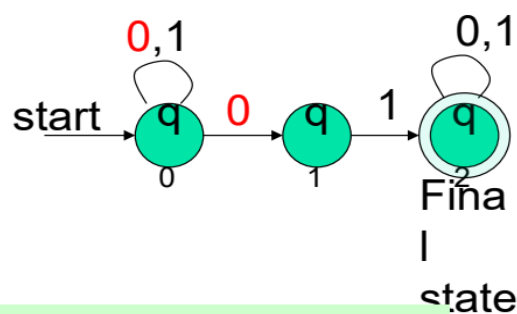
- If AT LEAST ONE current state is final: ACCEPT
- Otherwise: REJECT

Key Difference from DFA:

- DFA: Track ONE current state
- NFA: Track SET of current states

8.3 Example: NFA for Strings Containing "01"

Why is this non-deterministic?



What will happen if at state q_1 an input of 0 is received?

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1\}$
- start state = q_0
- $F = \{q_2\}$
- Transition table

δ	symbols	
	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	Φ	$\{q_2\}$
$*q_2$	$\{q_2\}$	$\{q_2\}$

Why This is Non-Deterministic: At state q_0 with input '0', machine can go to BOTH q_0 AND q_1 .

Execution Example: Input "01"

START: Current states = $\{q_0\}$

READ '0':

From q_0 on '0': $\{q_0, q_1\}$

Current states = $\{q_0, q_1\}$

READ '1':

From q_0 on '1': $\{q_0\}$

From q_1 on '1': $\{q_2\}$

Union: $\{q_0, q_2\}$

Current states = $\{q_0, q_2\}$

END: Current states = $\{q_0, q_2\}$

Is any state in F? YES (q_2 is final)

Result: ACCEPT ✓

8.3 Example: Build an NFA for the following language: $L = \{w \mid w \text{ ends in } 01\}$

Ans:



9. DFA vs NFA Comparison

9.1 Deterministic vs Non-Deterministic Automata

A Deterministic Finite Automaton (DFA) has exactly one path to follow for each input symbol. From any state, for every symbol, there is only one possible next state. This makes the machine's behavior predictable, straightforward, and easy to trace. DFA has no ϵ -transitions and always has a single current state while reading input.

In contrast, a Non-Deterministic Finite Automaton (NFA) may have multiple possible paths for the same input symbol—or even ϵ -moves that require no input. Instead of one current state, an NFA can be in many states at the same time. A string is accepted if any one of the possible paths leads to a final state.

Although NFAs seem more flexible, both DFA and NFA recognize exactly the same class of languages (regular languages).

Equivalence of DFA and NFA

Theorem:

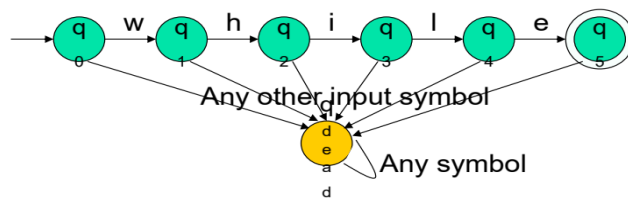
A language L is accepted by a DFA **if and only if** it is accepted by an NFA.

Explanation:

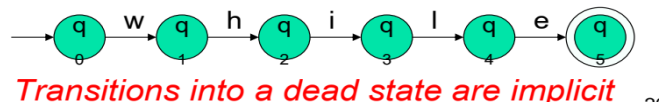
- **Only-if part:** Every DFA is a special case of an NFA (each state has exactly one transition per symbol). Hence, any language accepted by a DFA is also accepted by an NFA.
- **If part:** For every NFA, we can construct an equivalent DFA using **subset construction**, where each DFA state represents a set of NFA states. The resulting DFA accepts the same language.

9.2 Visual Comparison

A DFA for recognizing the key word
“while”



■ An NFA for the same purpose:



9.3 Important Theorem

THEOREM: DFA and NFA are equivalent in power.

Meaning:

- Any language accepted by an NFA can be accepted by some DFA
- Any language accepted by a DFA can be accepted by some NFA
- Both recognize exactly the **Regular Languages**

Formula: DFA $\equiv$ NFA (in terms of language recognition)

Dead State (Definition)

A dead state is a state in a DFA where: Once you enter it, you can never reach a final (accepting) state again—no matter what input you read.

Language of an NFA (Standard Definition)

An NFA accepts a string **w** if **there exists at least one path** from the start state q_0 to a final (accepting) state that is labeled by w .

Formal Definition

$$L(N) = \{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset \}$$

Meaning in simple words

- Start from the **start state** q_0
- Read the string w
- The NFA may go to **many states**
- If **at least one** of those states is a **final state**, the string is accepted

9.4 Advantages & Caveats of NFA

Advantages

- NFAs are **very useful for modeling regular expressions**.
- They are widely used in **string processing applications**, such as:
 - grep
 - lexical analyzers (tokenizers in compilers)
- NFAs often provide **simpler and more compact designs** than equivalent DFAs.

Caveats / Limitations

- A true NFA **cannot be directly implemented** on a single, sequential computer.
- Conceptually, an NFA can be thought of as being in **multiple states at the same time**.
- This behavior could be imagined on a **parallel computer**, where all paths are explored simultaneously.

9.5 Difference between DFA and NFA

■ DFA

1. All transitions are deterministic
 - Each transition leads to exactly one state
2. For each state, transition on all possible symbols (alphabet) should be defined
3. Accepts input if the last state is in F
4. Sometimes harder to construct because of the number of states
5. Practical implementation is feasible

■ NFA

1. Some transitions could be non-deterministic
 - A transition could lead to a subset of states
2. Not all symbol transitions need to be defined explicitly (if undefined will go to a dead state – this is just a design convenience, not to be confused with “non-determinism”)
3. Accepts input if *one of* the last states is in F
4. Generally easier than a DFA to construct
5. Practical implementation has to be deterministic (convert to DFA) or in the form of parallelism

31

10. Converting NFA to DFA (Subset Construction)

10.1. NFA to DFA Conversion (Subset Construction)

The Problem

We have an NFA and want to convert it to an equivalent DFA that accepts the same language.

Why convert?

- DFAs are easier to implement in hardware/software
- DFAs have deterministic execution (no guessing needed)
- DFAs are easier to analyze and minimize

The Subset Construction Algorithm

Given: NFA $N = \{Q_N, \Sigma, \delta_N, q_0, F_N\}$

Goal: Build DFA $D = \{Q_D, \Sigma, \delta_D, \{q_0\}, F_D\}$ such that $L(D) = L(N)$

Construction Steps:

Step 1: States (Q_D)

- $Q_D =$ all subsets of Q_N (the power set of Q_N)
- Each state in the DFA represents a **set of states** from the NFA
- If NFA has n states, DFA potentially has 2^n states

Step 2: Start State

- Start state of DFA = $\{q_0\}$ (singleton set containing NFA's start state)

Step 3: Final States (F_D)

- $F_D = \{S \subseteq Q_N \mid S \cap F_N \neq \emptyset\}$
- A DFA state S is final if it contains **at least one** NFA final state
- In other words: if any NFA state in the set is accepting, the DFA state is accepting

Step 4: Transition Function (δ_D)

- For each subset S of Q_N and each input symbol a in Σ :
- $\delta_D(S, a) = \bigcup \{p \in S \mid \delta_N(p, a) \in F_N\}$
- **Meaning:** From DFA state S , on input a , go to the union of all states reachable from any NFA state in S

Formula breakdown:

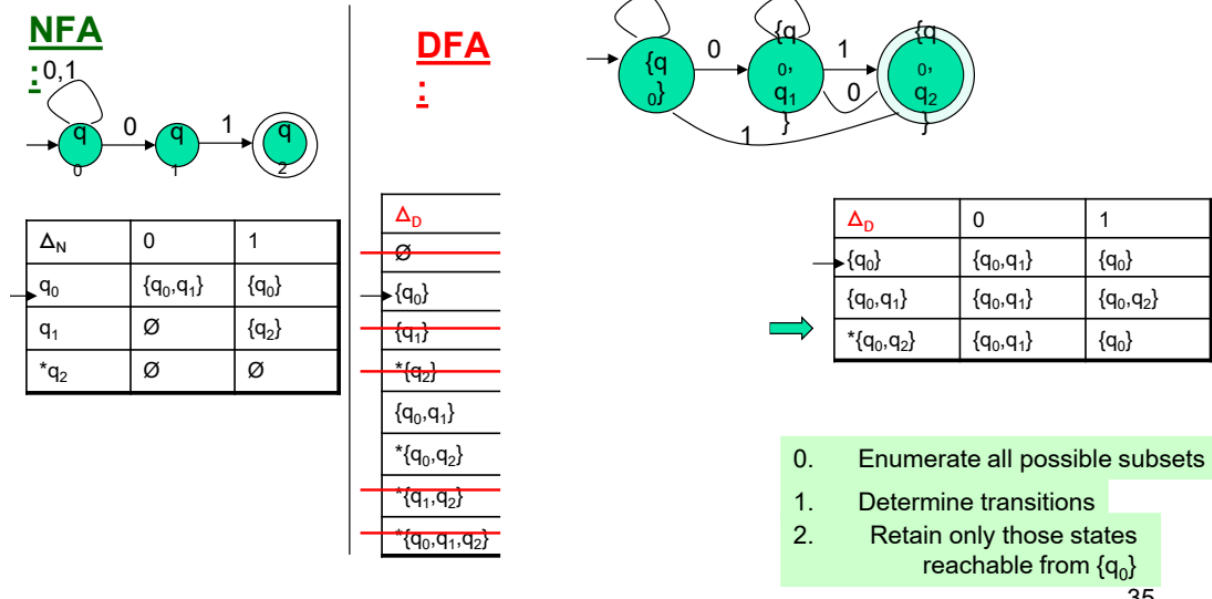
$$\delta_D(\{p_1, p_2, \dots, p_k\}, a) = \delta_N(p_1, a) \cup \delta_N(p_2, a) \cup \dots \cup \delta_N(p_k, a)$$

Complete Example: Language Ending in "01"

Language: $L = \{w \mid w \text{ ends in } 01\}$

Examples: 01, 001, 101, 0001, 1001, 00101

■ $L = \{w \mid w \text{ ends in } 01\}$



2. Lazy State Creation Method

The Problem with Full Enumeration

For an NFA with n states:

- Power set has 2^n states
- Most of these states are **unreachable** from the start state
- Computing transitions for all 2^n states is wasteful

Example: An NFA with 10 states would create $2^{10} = 1024$ DFA states, but often only 10-20 are actually reachable!

The Lazy Creation Algorithm

Main Idea: Create DFA states **on demand** as they become reachable.

4. Worst-Case Exponential Blowup(Never Read)

The Problem

Sometimes the DFA really does need exponentially many states compared to the NFA.

Example: "kth Symbol from End is 1"

Language: $L = \{w \mid \text{the } k\text{th symbol from the end of } w \text{ is } 1\}$

For $k = 2$: $\{w \mid \text{the second-to-last symbol is } 1\}$ Examples: 10, 11, 010, 011, 110, 111, 0010, 0011, ...

NFA Construction ($k+1$ states):

States: $q_0, q_1, q_2, \dots, q_k$

Transitions:

$q_0 \xrightarrow{0,1} q_0$ (read any symbols)

$q_0 \xrightarrow{1} q_1$ (guess: this is the k th from end!)

$q_1 \xrightarrow{0,1} q_2$

$q_2 \xrightarrow{0,1} q_3$

...

$q_{k-1} \xrightarrow{0,1} q_k(\text{final})$

For $k=2$:

$q_0 \xrightarrow{0,1} q_0$

$q_0 \xrightarrow{1} q_1$

$q_1 \xrightarrow{0,1} q_2(\text{final})$

This NFA has 3 states.

DFA Requirement:

Claim: Any DFA for this language needs at least 2^k states.

Intuition:

- The DFA must "remember" the last k symbols seen
- There are 2^k possible combinations of k binary symbols
- Each combination requires a different state

5. Epsilon-NFA (ϵ -NFA)

Definition

An ϵ -NFA is an NFA that allows **epsilon transitions** (ϵ -transitions):

- Transitions that move from one state to another **without consuming any input**
- Represented by ϵ (epsilon) or λ (lambda)

Formal Definition: ϵ -NFA = $\{Q, \Sigma, \delta, q_0, F\}$

- $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$
- The transition function now includes ϵ as a possible input

Why Use ϵ -NFAs?

Advantages:

1. **Easier to construct** for certain languages
2. **More intuitive** for combining automata
3. **Useful for proving equivalences**
4. **Simplifies RE to automaton conversion**

Example: Combining two NFAs with union or concatenation is simpler with ϵ -transitions.

Example ϵ -NFA

Language: $L = \{w \mid w \text{ is empty, or if non-empty, ends in } 01\}$

Examples: ϵ , 01, 001, 101, 1101, 00101

ϵ -NFA:

States: q'_0, q_0, q_1, q_2

Start: q'_0

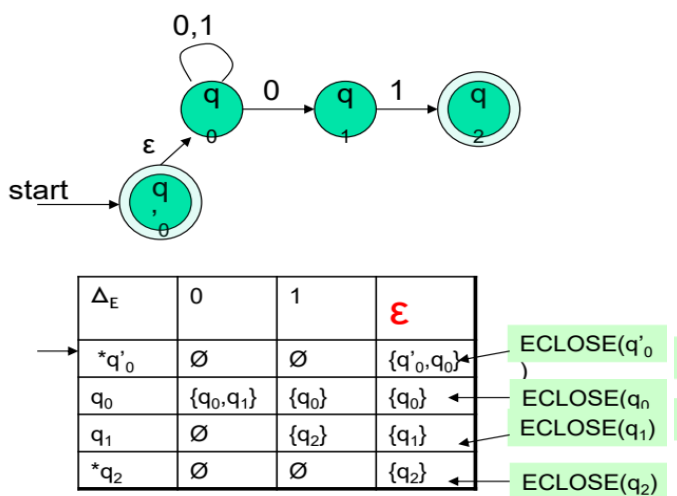
Final: q'_0, q_2

Transitions:

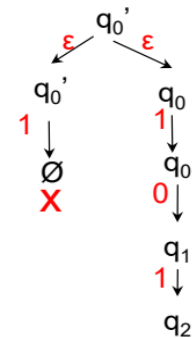
- From $q'_0 \rightarrow q_0$ using ϵ
 - This means the automaton can move to q_0 without reading any symbol.
- $\epsilon\text{-closure}(q'_0) = \{q'_0, q_0\}$
- $\epsilon\text{-closure}(q_0) = \{q_0\}$
- $\epsilon\text{-closure}(q_1) = \{q_1\}$
- $\epsilon\text{-closure}(q_2) = \{q_2\}$

These are shown in the ϵ column of the table.

Visual:



Simulate for $w=101$:



Simulating ϵ -NFA

Key Rule: When processing any transition:

Step 1: Go to all immediate destination states

Step 2: From there, follow **all ϵ -transitions** (take ϵ -closure)

Example Simulation: $w = "101"$

Initial: q'_0

Take ϵ -closure: $\{q'_0, q_0\}$ (follow ϵ from q'_0 to q_0)

Read '1':

From q'_0 : no transition on 1

From q_0 : goes to $\{q_0\}$

Take ϵ -closure of $\{q_0\}$: $\{q_0\}$

Current states: $\{q_0\}$

Read '0':

From q_0 : goes to $\{q_0, q_1\}$

Take ϵ -closure: $\{q_0, q_1\}$

Current states: $\{q_0, q_1\}$

Read '1':

From q_0 : goes to $\{q_0\}$

From q_1 : goes to $\{q_2\}$

Union: $\{q_0, q_2\}$

Take ϵ -closure: $\{q_0, q_2\}$

Current states: $\{q_0, q_2\}$

q_2 is final $\rightarrow$ ACCEPT $\checkmark$

6. Epsilon-Closure Concept

Definition

The **ϵ -closure** (or ECLOSE) of a state q is the set of all states reachable from q by following zero or more ϵ -transitions.

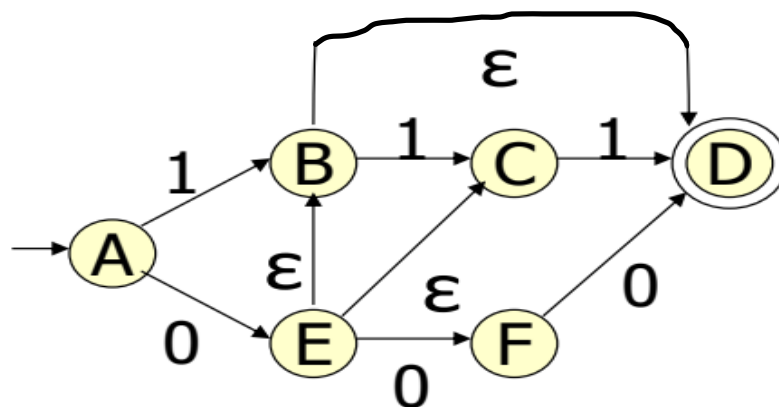
Formally: $ECLOSE(q) = \{p \mid \text{there exists a path from } q \text{ to } p \text{ using only } \epsilon\text{-transitions}\}$

Note: q is always in $ECLOSE(q)$ (zero ϵ -transitions).

How to find ϵ -closure

To find ϵ -closure(q):

1. Start with state q
2. Add q to the set
3. Follow only ϵ arrows
4. Add every state you can reach
5. Stop when no new state is found



Computing ϵ -Closures:

ϵ -closure(A)

- From A, there is no ϵ arrow
- So you cannot go anywhere without input

$$CL(A) = \{A\}$$

ϵ -closure(E)

Now look at state E:

1. Start at E $\rightarrow$ include E
2. From E, there is an ϵ arrow to B
3. From B, go to C using ϵ
4. From C, go to D using ϵ
5. No more ϵ moves

$$CL(E) = \{E, B, C, D\}$$

$$ECLOSE(B) = \{B\} \text{ (no } \epsilon\text{-transitions from B)}$$

$$ECLOSE(D) = \{D\}$$

$$ECLOSE(F) = \{F\} \text{ (self-loop doesn't add new states)}$$

ϵ -Closure of a Set

For a set of states S:

Example:

$$\epsilon\text{-closure}(\{A, E\})$$

$$= \epsilon\text{-closure}(A) \cup \epsilon\text{-closure}(E)$$

$$= \{A\} \cup \{E, B, C, D\}$$

$$= \{A, B, C, D, E\}$$

7. ϵ -NFA to DFA Conversion

The Algorithm

Given: ϵ -NFA $E = \{Q_E, \Sigma, \delta_E, q_0, F_E\}$

Goal: Build DFA $D = \{Q_D, \Sigma, \delta_D, q_D, F_D\}$ such that $L(D) = L(E)$

Construction:

Step 1: Start State

$$q_D = \text{ECLOSE}(q_0)$$

The DFA starts in the ϵ -closure of the NFA's start state.

Step 2: States (Q_D)

- Q_D = all reachable subsets of Q_E (considering ϵ -closures)
- Use lazy creation

Step 3: Final States (F_D)

$$F_D = \{S \subseteq Q_E \mid S \cap F_E \neq \emptyset\}$$

A DFA state is final if it contains at least one ϵ -NFA final state.

Step 4: Transition Function (δ_D)

For each subset S of Q_E and each input symbol $a \in \Sigma$:

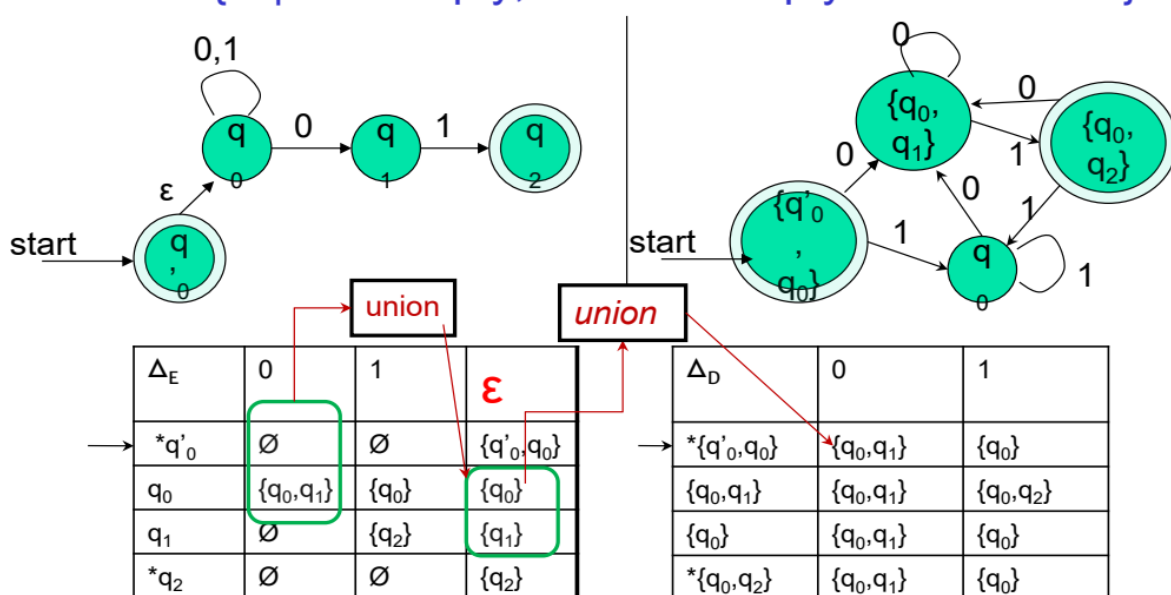
1. $R = \bigcup (p \in S) \delta_E(p, a)$ // Go to immediate destinations
2. $\delta_D(S, a) = \bigcup (r \in R) \text{ECLOSE}(r)$ // Take ϵ -closure

In words:

1. From all states in S , find where symbol 'a' takes you
2. From all those destination states, take the ϵ -closure

Complete Example: "Empty or Ends in 01"

$$L = \{w \mid w \text{ is empty, or if non-empty will end in } 01\}$$



Step 2: Build DFA using Lazy Creation

Process $\{q'_0, q_0\}$:

On input 0:

From q'_0 : $\delta_E(q'_0, 0) = \emptyset$

From q_0 : $\delta_E(q_0, 0) = \{q_0, q_1\}$

$R = \emptyset \cup \{q_0, q_1\} = \{q_0, q_1\}$

$ECLOSE(\{q_0, q_1\}) = ECLOSE(q_0) \cup ECLOSE(q_1)$

$= \{q_0\} \cup \{q_1\}$

$= \{q_0, q_1\}$

$\delta_D(\{q'_0, q_0\}, 0) = \{q_0, q_1\}$

On input 1:

From q'_0 : $\delta_E(q'_0, 1) = \emptyset$

From q_0 : $\delta_E(q_0, 1) = \{q_0\}$

$R = \{q_0\}$

$ECLOSE(\{q_0\}) = \{q_0\}$

$\delta_D(\{q'_0, q_0\}, 1) = \{q_0\}$

Process $\{q_0, q_1\}$:

On input 0:

From q_0 : $\delta_E(q_0, 0) = \{q_0, q_1\}$

From q_1 : $\delta_E(q_1, 0) = \emptyset$

$R = \{q_0, q_1\}$

$ECLOSE(\{q_0, q_1\}) = \{q_0, q_1\}$

$\delta_D(\{q_0, q_1\}, 0) = \{q_0, q_1\}$

On input 1:

From q_0 : $\delta_E(q_0, 1) = \{q_0\}$

From q_1 : $\delta_E(q_1, 1) = \{q_2\}$

$R = \{q_0, q_2\}$

$ECLOSE(\{q_0, q_2\}) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$

$\delta_D(\{q_0, q_1\}, 1) = \{q_0, q_2\}$

Process $\{q_0\}$:

On input 0:

$$\delta_E(q_0, 0) = \{q_0, q_1\}$$

$$ECLOSE(\{q_0, q_1\}) = \{q_0, q_1\}$$

$$\delta_D(\{q_0\}, 0) = \{q_0, q_1\}$$

On input 1:

$$\delta_E(q_0, 1) = \{q_0\}$$

$$ECLOSE(\{q_0\}) = \{q_0\}$$

$$\delta_D(\{q_0\}, 1) = \{q_0\}$$

Process $\{q_0, q_2\}$:

On input 0:

$$\text{From } q_0: \delta_E(q_0, 0) = \{q_0, q_1\}$$

$$\text{From } q_2: \delta_E(q_2, 0) = \emptyset$$

$$R = \{q_0, q_1\}$$

$$\delta_D(\{q_0, q_2\}, 0) = \{q_0, q_1\}$$

On input 1:

$$\text{From } q_0: \delta_E(q_0, 1) = \{q_0\}$$

$$\text{From } q_2: \delta_E(q_2, 1) = \emptyset$$

$$R = \{q_0\}$$

$$\delta_D(\{q_0, q_2\}, 1) = \{q_0\}$$

Final DFA:

DFA Transition Table:

State	0	1
* $\{q'_0, q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
* $\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

Final states: $\{q'_0, q_0\}, \{q_0, q_2\}$ (contain q'_0 or q_2)

Visual DFA:

$\{q'_0, q_0\}$ (start, final) $\xrightarrow{0} \{q_0, q_1\}$

$\{q'_0, q_0\} \xrightarrow{1} \{q_0\}$

$\{q_0, q_1\} \xrightarrow{0} \{q_0, q_1\}$

$\{q_0, q_1\} \xrightarrow{1} \{q_0, q_2\}$ (final)

$\{q_0\} \xrightarrow{0} \{q_0, q_1\}$

$\{q_0\} \xrightarrow{1} \{q_0\}$

$\{q_0, q_2\}$ (final) $\xrightarrow{0} \{q_0, q_1\}$

$\{q_0, q_2\}$ (final) $\xrightarrow{1} \{q_0\}$

8. Epsilon NFA to NFA conversion

Step-by-Step Rules

1. Copy outgoing transitions of S_2 to S_1

For every transition

$$S_2 \xrightarrow{a} X$$

add the transition

$$S_1 \xrightarrow{a} X$$

for the same input symbol a (do **not** change labels).

2. Initial-state rule

- If S_1 is an initial state, then S_2 also becomes an initial state.

3. Final-state rule

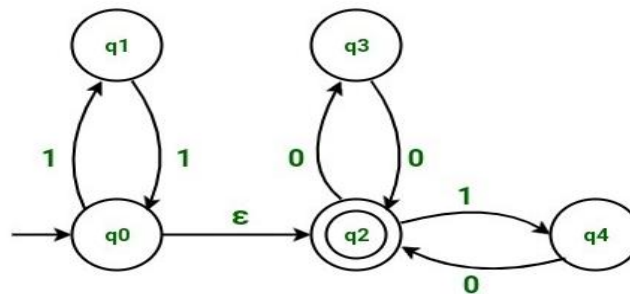
- If S_2 is a final (accepting) state, then S_1 also becomes a final state.

4. Remove the ϵ -transition

After applying the above steps, delete the ϵ -edge

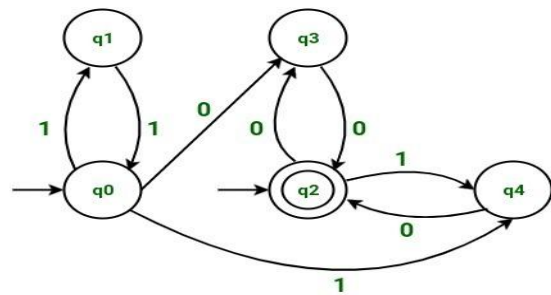
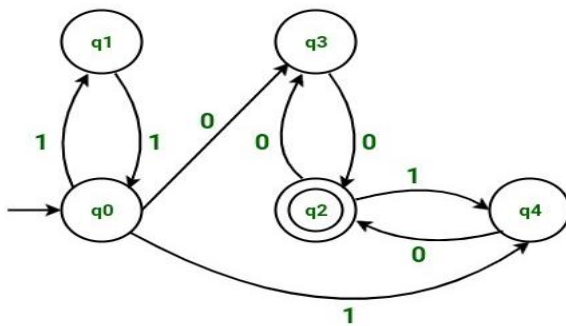
$$S_1 \xrightarrow{\epsilon} S_2$$

Example: Convert the following ϵ -NFA to NFA. The ϵ -NFA has states q_0, q_1, q_2, q_3, q_4 where q_0 is the initial state and q_2 final state.



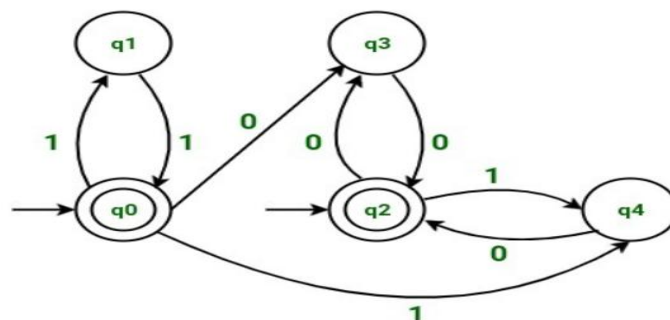
Transition table for the state transitions:

States/Input	Input 0	Input 1	Input epsilon
q0	-	q1	q2
q1	-	q0	-
q2	q3	q4	-
q3	q2	-	-
q4	q2	-	-



Step:01- Duplicate the moves

Step:02: Making state q_2 as a start state



Step:03: Resulting NFA (state q_0 as a final state)

9. Equivalence of DFA, NFA, and ϵ -NFA

The Grand Theorem

Theorem: A language L is accepted by some ϵ -NFA if and only if L is accepted by some DFA.

Corollary: $\text{DFA} \equiv \text{NFA} \equiv \epsilon\text{-NFA}$

All three types of automata accept exactly the **regular languages**.

Chain of Equivalences

$\epsilon\text{-NFA} \rightarrow \text{DFA}$ (proven by 7 algorithm) $\text{NFA} \rightarrow \text{DFA}$ (proven by subset construction) $\text{DFA} \rightarrow \text{NFA}$
(trivial: every DFA is an NFA) $\text{DFA} \rightarrow \epsilon\text{-NFA}$ (trivial: every DFA is an ϵ -NFA)

Therefore: All are equivalent in expressive power!

Why Multiple Models?

If they're all equivalent, why have different models?

ϵ -NFA:

- Easiest to construct from regular expressions
- Most intuitive for combining automata
- Useful in compiler design

NFA:

- More compact than DFA
- Easier to understand conceptually
- Good for pattern matching

DFA:

- Fastest to execute (no backtracking)
- Easiest to implement in hardware
- Unique minimal form (canonical)
- Best for actual implementation

